Implementação de um parser descendente recursivo

Objetivo: receber tokens, reconhecer programas da MiniLang e construir uma Árvore Sintática Abstrata (AST).

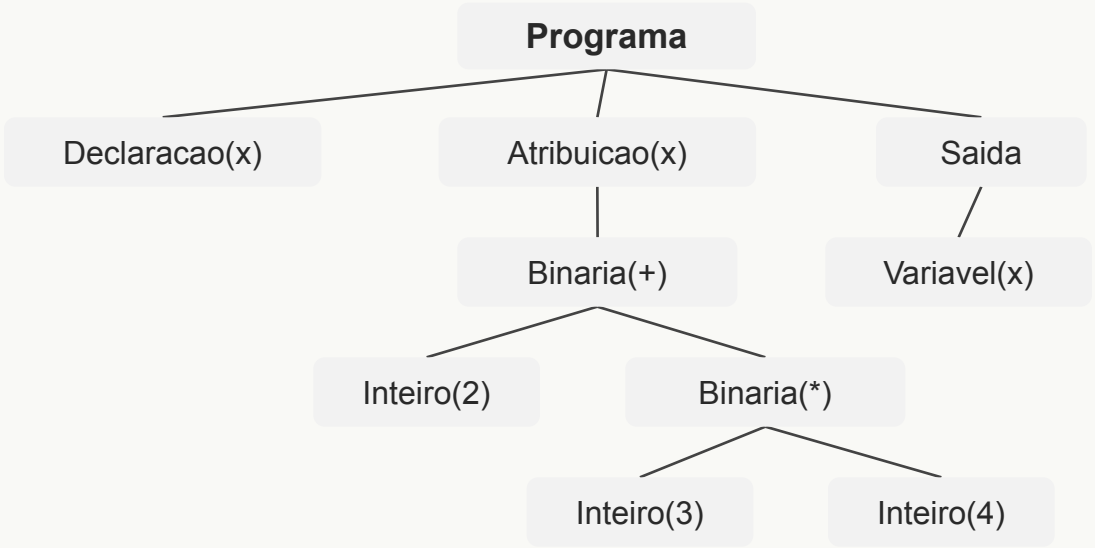


Do código-fonte à AST

PROGRAMA

```
int x;
x = 2 + 3 * 4;
print(x);
```

AST ESPERADA



A AST representa a estrutura do programa. O parser não executa operações nem verifica declarações de variáveis.

O que o parser deve reconhecer?

OBRIGATÓRIO

- Declarações int
- Atribuições
- Comando print
- Identificadores e literais inteiros
- Operadores +, -, * e /
- Expressões com parênteses
- Sequência de comandos
- Precedência e associatividade à esquerda
- Fim completo da entrada (EOF)

FORA DO ESCOPO

- if, else e while
- Blocos e escopos aninhados
- Booleanos e operadores relacionais
- Operadores unários
- Análise semântica
- Execução do programa

Utilizar a gramática da MiniLang definida nas aulas anteriores.

O que já está pronto?

Recurso	O que já está pronto
Lexer	Lexer de referência e estrutura Token
AST	Classes da AST e visualizador textual
Operações	peek(), advance(), expect(), match() e at_end()
Referências	declaracao(), fator(), termo() e saida()
Testes	Testes iniciais e programas de exemplo

Não é necessário implementar novamente o lexer ou as classes da AST.

O que vocês devem implementar?

Etapa 1 Expressões	→	Etapa 2 Comandos e programa	→	Etapa 3 Diagnósticos
Implementar expressao(). Reconhecer + e -. Construir nós Binaria. Preservar precedência e associatividade.		Implementar atribuicao(), item(), lista_itens() e programa(). Construir Programa com os comandos reconhecidos.		Identificar entradas inválidas. Informar token encontrado e posição. Parar no primeiro erro sintático.

Implemente uma etapa, execute os testes e depois avance para a próxima.

Como verificar o funcionamento?

Entrada	Resultado esperado
`print(2 + 3 * 4);`	Soma na raiz, produto à direita
`print((2 + 3) * 4);`	Produto na raiz, soma à esquerda
`print(10 - 3 - 2);`	Agrupamento (10 - 3) - 2
`print(2 + );`	Erro sintático
`print(y);`	Sintaticamente válido
`Entrada vazia`	Programa([])

Verifique a estrutura da AST, o consumo dos tokens e os diagnósticos. Não execute as expressões.

Entrega do Trabalho 02

ENTREGÁVEIS

- Código do parser implementado
- Testes executados
- Exemplo de AST produzida
- Exemplo de diagnóstico sintático
- Breve README com instruções de execução

CRITÉRIOS DE AVALIAÇÃO

- Reconhecimento correto dos programas
- Estrutura da AST
- Precedência e associatividade
- Diagnósticos com posição
- Testes e organização do código